

DriftNet: A Dual-Head Trajectory Transformer for Detecting and Localizing Prompt Injection in LLM Agents

Asif Pinjari
*School of Informatics, Computing,
and Cyber Systems*
Northern Arizona University
Flagstaff, AZ, USA
ap3929@nau.edu

Mithun Paul Saint-Germain
*School of Informatics, Computing,
and Cyber Systems*
Northern Arizona University
Flagstaff, AZ, USA
mithun.paul@nau.edu

Abstract—When an indirect prompt injection succeeds against an LLM agent, the compromise is visible in the agent’s own behavior: a benign prefix of tool calls, a poisoned observation, and a suffix of actions that serve the attacker. An operator needs three facts: where the attack entered, which steps it corrupted, and whether apparent poison was resisted. Existing systems return either a whole-trace verdict or a single unsafe index. We present DriftNet, a dual-head trajectory Transformer that reads a logged tool-call trajectory and answers all three questions in one forward pass: one head classifies the trajectory as compromised or not, and a second assigns every step one of four labels (benign, injection point, hijacked, failed injection). To our knowledge it is the first supervised detector to produce this joint output. A frozen sentence encoder and four identity-free world features embed each step; the trained trunk, under two million parameters and optimized with a class-weighted joint objective over both heads, needs no access to the agent’s model. On the task-disjoint split of the AgentDrift benchmark (12,536 trajectories, 71,024 labeled steps), with a 20-configuration sweep bounding hyperparameter sensitivity to 0.011 F1 and the test part evaluated exactly once, DriftNet reaches trajectory-level F1 of 0.983, exact injection-point recovery on 98.7% of attacked trajectories, hijacked-span IoU of 0.979, zero flags on 218 resisted attacks, and 2.9% flags on hard negatives. A surface baseline retrained on the identical split recovers 11.1% of partial hijacks and 17.1% of delayed executions; DriftNet reaches 98.6% and 93.2% while lowering every false-alarm rate. Reading all 26 residual errors shows that most misses trace to trajectories whose labeled injection observation carries no legible instruction, and we report the benchmark’s measured world-identity regularity alongside the results.

Index Terms—LLM agents, prompt injection, trajectory analysis, sequence labeling, anomaly detection, agent security

I. INTRODUCTION

LLM agents act on the world by interleaving reasoning with tool calls. Following the pattern popularized by ReAct [1], every step couples a reasoning thought with a tool invocation, its arguments, and the returned observation, and such agents already read inboxes, move money, browse the web, edit code, and retrieve patient records. Every observation the agent reads is also an attack surface. Indirect prompt injection [2] hides instructions in content the agent will later retrieve;

since a language model draws no hard line between data and instructions, the agent can adopt the planted goal as its own. The threat is measured, not hypothetical: a ReAct-prompted GPT-4 follows injected instructions in roughly a quarter of InjecAgent’s test cases [3], Agent Security Bench reports attack success above 80% for some backbones [4], and stage-level tracking shows injected payloads surviving across surfaces and agent boundaries in multi-agent pipelines [5].

When an injection succeeds, the compromise has a shape. Execution begins benignly, a poisoned observation arrives mid-task, and the steps that follow drift away from the user’s goal toward the attacker’s. Detecting this is a sequence problem: each step must be judged against the task, the world the agent operates in, and everything that came before. Localizing it is what makes detection actionable. An operator confronted with a flagged trajectory needs to know the step where the attack entered, the span of actions it corrupted, and whether apparent poison was actually resisted, because those three facts decide what to roll back, what to audit, and which content source to distrust.

Existing systems do not produce this output. Live-loop defenses veto or re-derive single actions and assume control of the running agent [6]–[9]; internal-state probes and attention localizers require white-box access to the agent’s model [10]–[12]. Detectors that do read completed trajectories emit either a single verdict on the whole trace, with resisted injections folded into the safe class [13]–[15], or a single index: the first unsafe action [16], the one mutated step [17], the divergence onset [18], the error step [19], or the root-cause component [20]. A single index cannot represent an agent that complied for two steps and recovered, an execution delayed past benign steps, or an injection that was seen and refused. What has been missing is a detector whose output is the full picture: a trajectory verdict together with an injection-specific label on every step.

This paper presents DriftNet, a detector with exactly that output, trained and evaluated on the AgentDrift benchmark [21], whose 71,024 step labels make dense, injection-

specific supervision of this task available for the first time. DriftNet is deliberately small. Each step is serialized to text and embedded by a frozen sentence encoder; four world-grounded, identity-free features recover what semantics cannot see, namely whether the step’s action targets a recipient outside the user’s known world; and a Transformer encoder of at most three layers reads the sequence with two heads, one pooling into a trajectory verdict, one labeling every step as *benign*, *injection_point*, *hijacked*, or *failed_injection*. The trained trunk is under two million parameters, three orders of magnitude below contemporary LLM-scale guards [16], and it consumes only the logged trajectory: no agent internals, no re-execution, no live loop.

The evaluation is designed to be hard to fool and easy to trust. All experiments use the corpus’s task-disjoint split, in which no task template is shared between training and test, so template memorization cannot inflate the numbers. A 20-configuration random sweep establishes that the result does not depend on a lucky configuration (every draw lands within a 0.011 band of validation F1). The held-out test part is evaluated exactly once. And the surface baseline we compare against is retrained on the identical split, making the comparison like for like. Under this protocol DriftNet reaches trajectory-level F1 of 0.983; it recovers the exact injection-point set in 98.7% of attacked trajectories and the hijacked span at mean IoU 0.979; it flags zero of 218 resisted attacks and 2.9% of hard negatives. Where the surface baseline collapses on the benchmark’s two stealthy compliance patterns (11.1% recall on partial hijacks, 17.1% on delayed executions), DriftNet reaches 98.6% and 93.2%. Reading all 26 residual errors individually shows that they are confident rather than marginal, that the misses concentrate on delayed executions, and that in nine of twelve misses the labeled injection observation contains no legible instruction at all, pointing to residual generation noise in the corpus rather than to evidence the detector failed to read.

We also state plainly what these numbers do not establish. The corpus is synthetic and single-generator, and the dataset paper measures a world-identity regularity in it that a lookup exploits to 86% binary accuracy; our world features are immune by construction, but the text embeddings are not, and Section X discusses what that does and does not explain. The claim of this paper is scoped accordingly: on the first benchmark dense enough to supervise the task, joint trajectory classification and four-way step labeling is learnable to high fidelity by a small, cheap, model-agnostic detector.

Our contributions are as follows.

- **DriftNet.** A dual-head trajectory Transformer that, to our knowledge, is the first supervised detector to jointly classify a tool-call trajectory and label every step four ways, locating the injection point, the hijacked span, and resisted injections in one forward pass from the log alone (Section V).
- **Strict localization metrics.** Injection-point exact-set match and hijacked-span IoU, which expose localization failures

that per-class F1 hides (Section III-D).

- **A trust-preserving protocol.** Task-disjoint training, a robustness sweep, a single test evaluation, and a like-for-like retrained baseline (Section VI).
- **Results and analysis.** High-fidelity detection and localization on AgentDrift with pattern, domain, and family breakdowns; an exhaustive error analysis with a calibration caution; and an honest account of the benchmark’s measured artifacts (Sections VII to X).

The remainder of the paper proceeds from related work (Section II) through formulation (Section III), the benchmark and split (Section IV), the architecture (Section V), and the protocol (Section VI), to results (Section VII), error analysis (Section VIII), discussion (Section IX), limitations (Section X), and future work (Section XI).

II. RELATED WORK

A. Indirect Prompt Injection and Agent Attack Benchmarks

Prompt injection was first characterized as an attack class on instruction-following models [22] and then shown to compromise deployed LLM-integrated applications through content the model retrieves rather than through the user’s prompt [2]. Liu et al. formalized the attack family and benchmarked defenses at the prompt level [23]. For tool-using agents the threat is measured by live attack benchmarks: InjecAgent reports that a ReAct-prompted GPT-4 follows injected instructions in roughly a quarter of cases [3], AgentDojo executes attacks inside a stateful tool environment [24], Agent Security Bench reports attack success above 80% for some backbones [4], and AgentDyn moves the evaluation to dynamically generated environments [25]. Recent work broadens the measurement surface: GuardianAgentBench evaluates 580 scenarios across three production agent frameworks under five adversarial modes [26], and kill-chain instrumentation tracks a canary payload stage by stage across six attack surfaces to separate where an injection is neutralized from where it merely fails to execute [5]. Surveys catalog the widening gap between agent capability and agent security [27]. All of these resources measure whether attacks succeed against a live agent; none of them trains or evaluates a detector that reads a completed trajectory, which is the setting of this paper.

B. Runtime Defenses at the Input and Action Level

A second line of work intervenes in the live agent loop. Architectural defenses constrain what retrieved content can do, from control- and data-flow separation in CaMeL [28] to tool-dependency graphs that pin the plan before untrusted content is read [7]. MELON detects injection by comparing the agent’s next action with and without the user task, exploiting the fact that a hijacked action is predictable from tool outputs alone [6]. AttriGuard asks why a tool call was produced, re-executing the agent under control-attenuated views of its observations to test whether the call is causally driven by untrusted content [8]. DreamGuard maintains a risk-aware world model over the trajectory and predicts hazard before execution [9]. Guard agents wrap the target agent in a reasoning

monitor [29], graph monitors watch multi-agent systems [30], moderation classifiers screen prompts and outputs [31], [32], and WebSentinel detects and localizes injected content inside retrieved web data [33]. These defenses assume control of the running agent: they can pause it, replay it, or veto its next action. DriftNet assumes strictly less. It reads a recorded trajectory and its world context, which makes it applicable to any agent whose tool calls are logged, including after the fact in forensic triage, but it cannot block an action before execution. The two settings are complementary rather than competing.

C. Internal-State Probes and Attention Localization

A third line reads the model’s internals rather than its behavior. TaskTracker catches the model’s task drifting under injected instructions by reading activation deltas [34], linear probes on pre-generation hidden states predict injection exposure with AUROC above 0.90 across eight models [10], and BASIS trains attention probes that separate inputs the model would resist from inputs that would actually breach it, refusing only the latter [11]. Its breach-versus-resisted distinction at the input level parallels our `failed_injection` class at the behavioral level: both refuse to equate the presence of an attack with its success. AttnLocate localizes the context spans that actually drive a tool-calling decision by treating attention matrices as an object-detection input [12]. A cautionary study shows that near-perfect probe AUROC can reflect nuisance correlations rather than detection of malicious content, and argues for controlled evaluation of such probes [35]. All of these require white-box access to the agent’s model at inference time. DriftNet is model-agnostic by construction: it never sees the agent’s weights, activations, or attention, only the logged trajectory, so it applies unchanged to closed-source agents.

D. Trajectory Guard Models and Step-Level Resources

Closest to this paper are detectors and datasets that judge completed trajectories. LLM judges score full traces for safety [13], and trajectory-level guard benchmarks report that even frontier models reach only 76.7% F1 on binary trace safety [14]. ATBench states explicitly that it contains no step-level labels, and AgentDoG labels a trajectory safe when the agent resisted an injection, collapsing the resisted class [14], [15]. TraceAegis mines behavioral hierarchies from normal logs for anomaly detection [36], and a lightweight sequence model detects non-adversarial plan anomalies at the trajectory level [37]. Step-granular resources exist for other failure types: AgenTracer attributes procedural failures [38], Who&When benchmarks failure attribution in multi-agent runs [39], TrajAD localizes a single error step in agent mistakes [19], and StepShield marks the onset step of rogue behavior that originates in the agent itself [18]. TraceSafe injects a risk into exactly one step of a static trace, only two of its twelve risk categories being prompt injection [17], and NVIDIA packages synthetic injection environments as reinforcement-learning training data for injection-resistant agent policies [40].

Two concurrent efforts come nearest. StepGuard trains a 4B-parameter guard by reinforcement learning to emit a binary safe or unsafe judgment per action, with the first unsafe action as the supervision anchor [16]. Trajectory attribution ranks the components of a long-horizon trajectory by their causal contribution to an observed behavior, recovering a single primary root cause [20].

E. Positioning

Table I places DriftNet in this landscape along the axes that matter for injection triage. Every prior system outputs either a verdict (on the trace, the input, or the next action) or a single index or span (the first unsafe action, the mutated step, the root-cause component, the influential context span). None of them produces what an operator rolling back a compromised agent needs: a decision at the trajectory level together with a dense, injection-specific label on every step that separates the entry point of the attack from the span it corrupted and both from injections that were resisted. To our knowledge, DriftNet is the first supervised detector that jointly classifies the full trajectory and labels every step of it four ways, locating the injection point, the hijacked span, and resisted injections in one forward pass, and it does so from the logged trajectory alone, with no access to the agent’s model. This claim is scoped to the label structure and the joint task, not to the idea of step-level safety judgment in general, which StepGuard and StepShield also pursue with coarser outputs.

Two unrelated papers share the AgentDrift name, and we disambiguate them once. One studies behavioral degradation of multi-agent systems over extended interactions [41]; the other’s arXiv listing carries the same name for a study of unsafe recommendation drift under corrupted tool data in financial advisory agents, although the paper’s own title page reads differently [42]. Neither concerns injection detection or step labeling. Throughout this paper AgentDrift refers to the benchmark of Pinjari and Saint-Germain [21], on which this work trains, and DriftNet names the detector.

III. PROBLEM FORMULATION

A. Trajectories and World Context

A trajectory is an ordered sequence of steps

$$x = (s_1, s_2, \dots, s_T), \quad 3 \leq T \leq 11, \quad (1)$$

where each step

$$s_t = (\text{tool}_t, \text{thought}_t, \text{args}_t, \text{obs}_t) \quad (2)$$

records the tool the agent invoked, its stated reasoning, the arguments it passed, and the observation the tool returned. Each trajectory is accompanied by a world context $\mathcal{W}$: the user, their organization, and a contact list with names, email addresses, and relations. The world context is what grounds legitimacy. Whether forwarding a document is routine collaboration or exfiltration depends on whether the recipient appears in $\mathcal{W}$, and that fact is not recoverable from the step text alone [21].

TABLE I

POSITIONING AGAINST PRIOR TRAJECTORY-SECURITY SYSTEMS. ENTRY POINT: DOES THE SYSTEM IDENTIFY THE STEP OR SPAN THROUGH WHICH THE INJECTION ENTERED? CORRUPTED SPAN: DOES IT MARK EVERY SUBSEQUENT STEP THE ATTACK CORRUPTED? RESISTED CLASS: DOES IT DISTINGUISH ATTACKS THAT WERE PRESENT BUT NOT OBEYED? LOG ONLY: DOES IT OPERATE ON A RECORDED TRAJECTORY WITHOUT MODEL INTERNALS OR A LIVE AGENT LOOP?

System	Decision output	Per-step labels	Entry point	Corrupted span	Resisted class	Log only
Trajectory guards [13]–[15]	trace verdict	×	×	×	×	✓
StepGuard [16]	per-action safe/unsafe	binary	○ (first unsafe action)	×	×	×
TraceSafe [17]	trace verdict + risk type	×	○ (one mutated step)	×	×	✓
StepShield [18]	divergence onset	×	○ (onset index)	×	×	✓
TrajAD [19]	anomaly type + error step	×	○ (one index)	×	×	✓
Trajectory attribution [20]	root-cause ranking	×	✓ (primary component)	○ (chain)	×	✓
AttnLocate [12]	span adjudication	×	✓ (context span)	×	×	×
BASIS [11]	input breach verdict	×	×	×	✓ (input level)	×
Hidden-state probes [10], [35]	exposure verdict	×	×	×	×	×
DriftNet (this work)	trajectory verdict	four-way	✓ (injection_point)	✓ (hijacked)	✓ (failed_injection)	✓

B. Threat Model

The attacker’s channel is tool-returned content. In indirect prompt injection the adversary plants an instruction inside data the agent will retrieve during normal execution (an email body, a record field, a web page), so the injected instruction arrives as part of an observation, never as part of the user’s request [2], [3]. The attacker does not modify the user’s instruction, the agent’s weights, or the world context. The attack succeeds if the agent adopts the planted goal and its subsequent actions serve the attacker; the agent may instead recognize the instruction and resist it, in which case the poison text remains in the trajectory but behavior never deviates.

The defender observes a recorded trajectory together with its world context, either after execution or as a monitor running alongside it, with no access to the agent’s internals and no ability to re-execute it. This is a deliberately weak vantage point. Defenses that control the live agent loop [6]–[8] or read the model’s activations [10], [11] assume strictly more; a detector that works from the log alone applies to any agent whose tool calls are recorded, including closed-source ones. The threat model forces two distinctions on the detector: an attempted injection is not a successful one, and suspicious-looking legitimate content is not an attack.

C. Prediction Tasks

We pose two prediction problems over x , solved jointly.

1) *Trajectory-level detection*: Predict $y \in \{0, 1\}$, whether the trajectory is compromised by a prompt injection the agent acted on.

2) *Dense step labeling*: Predict a label for every step,

$$z_t \in \{B, I, H, F\}, \quad t = 1, \dots, T, \quad (3)$$

where B (benign) marks a step serving the user’s task, I (injection_point) the step whose observation carries the injected instruction, H (hijacked) a step whose action serves the injected goal, and F (failed_injection) a step carrying an injection the agent resisted. Ground-truth label strings $z_{1:T}$ belong to a regular grammar fixed by the benchmark: benign trajectories realize B^+ , full hijacks B^+IH^+ , partial hijacks $B^+IH^{1..2}B^+$, delayed executions $B^+IB^+HB^+$, and failed attacks B^+FB^+ [21].

The joint task is what gives a detection its operational value. A trajectory flag alone tells an operator that something went wrong; the step labels tell them where the compromise entered, which actions to roll back, and which content source to distrust.

D. Metrics

Trajectory-level detection is scored with precision, recall, and F1, with compromised trajectories as the positive class, plus the flag rate per true category (the fraction of each category’s trajectories flagged), which for the attacked category equals recall and for the other categories is a false-alarm rate. Step labeling is scored with per-class precision, recall, and F1 over the four labels, padding excluded.

Localization quality is scored by two measures computed on attacked trajectories. Let $I = \{t : z_t = I\}$ and $\hat{I} = \{t : \hat{z}_t = I\}$ be the gold and predicted injection-point sets, and $H, \hat{H}$ the corresponding hijacked sets. Injection-point exact match is

$$EM_I = \mathbf{1}[\hat{I} = I], \quad (4)$$

and hijacked-span overlap is the Jaccard index

$$IoU_H = \frac{|\hat{H} \cap H|}{|\hat{H} \cup H|}, \quad (5)$$

defined as 1 when both sets are empty. Both are averaged over attacked test trajectories. These measures are stricter than per-class F1: a detector can score high step F1 while consistently missing the entry point by one step, and EM_I exposes exactly that.

E. The Drift Hypothesis

The formulation rests on a drift hypothesis. A successful injection produces a benign prefix, an injection point riding in on a tool observation, and subsequent steps that drift away from the user’s task toward the attacker’s goal. The discriminating signal is not a single anomalous token but a deviation of the action sequence from the task, judged against the world context; MELON’s observation that a hijacked next action becomes predictable from tool outputs alone is the same signal read from inside the live loop [6]. Two consequences follow.

First, the model must see the whole ordered sequence, because whether step t is hijacked depends on the task established earlier and the observation that arrived before it. Second, the model must resist two traps: trajectories whose legitimate content merely looks suspicious, and trajectories containing real injected text the agent refused to follow. Both traps defeat detectors that fire on surface novelty.

F. Scope

We scope the task as detection plus localization, not attack-type classification. The benchmark releases attack-goal tags as generation metadata but does not certify them as classification targets, because semantically distinct goals partially converged during corpus construction [21]; we use those tags only descriptively, in the per-family recall analysis of Section VII-F.

IV. BENCHMARK AND SPLIT

A. The AgentDrift Corpus

We train and evaluate on AgentDrift [21], a benchmark of 12,536 synthetic tool-call trajectories over five agent domains (email, banking, web, coding, medical) with 71,024 individually labeled steps. Table II summarizes composition. Every trajectory belongs to one of four categories: benign executions, successful attacks, failed attacks in which the agent saw the injection and resisted it, and hard negatives whose legitimate content superficially resembles an attack (a security-warning email that a benign agent must handle, an audit address inside the user’s own domain). Attacked trajectories span six attack-goal families and three compliance patterns: full hijack, in which every step after the injection serves the attacker; partial hijack, in which the agent complies for one or two steps and then recovers; and delayed execution, in which the agent continues its task and acts on the injection several steps later. All trajectories were generated by Llama-3.3-70B-Instruct served through the AI-VERDE gateway [43], under a labeling protocol enforced by a closed-vocabulary structural validator, reviewed by an LLM judge, and audited by hand on more than 1,200 trajectories, with estimated label correctness of 99.6% [21]. We refer the reader to the dataset paper for generation details and worked examples. The corpus, including the task-disjoint split used throughout this paper, is publicly released under CC BY 4.0.¹

B. Task-Disjoint Split

The corpus distribution ships two splits: the stratified split whose statistics the dataset paper reports, and a task-disjoint split in which no task template is shared between train, validation, and test. All experiments in this paper use the task-disjoint split: 9,081 training, 1,733 validation, and 1,722 test trajectories. Task templates are the corpus’s main axis of repetition, so holding them out is the discipline that separates generalization from template memorization; a detector could otherwise score well by recognizing the task rather than the

TABLE II
AGENTDRIFT COMPOSITION [21].

Trajectory category	Trajectories	Share
benign	4,000	31.9%
attacked	5,536	44.2%
failed_attack	1,500	12.0%
hard_negative	1,500	12.0%
Total	12,536	100%
Step label	Steps	Share
benign	51,639	72.7%
injection_point	5,536	7.8%
hijacked	12,349	17.4%
failed_injection	1,500	2.1%
Total	71,024	100%

TABLE III
CATEGORY COMPOSITION OF THE TASK-DISJOINT VALIDATION AND TEST PARTS.

Category	Validation	Test
attacked	753	775
benign	584	525
failed_attack	193	218
hard_negative	203	204
Total	1,733	1,722

attack. Table III gives the category composition of the validation and test parts and also supplies the denominators for every per-category rate reported later. The test part contains 9,796 labeled steps: 7,048 benign, 775 injection_point, 1,755 hijacked, and 218 failed_injection.

V. DRIFTNET

Figure 1 shows the architecture. The design principle is to keep every expensive component frozen and train a small sequence model on top: a frozen sentence encoder turns each step into a vector, four world-grounded features restore what semantics cannot see, and a compact Transformer encoder with two heads reads the resulting sequence.

A. Input Representation

Each step is serialized to a single string in the fixed format

```
TOOL: {tool} | THOUGHT: {thought} | ARGS:
      {args} | OBS: {obs}
```

and embedded with the frozen sentence encoder all-mpnet-base-v2 [44], [45], yielding a normalized $e_t \in \mathbb{R}^{768}$. Embeddings are precomputed once per split and cached, which makes every training run cheap and its inputs deterministic.

Semantics alone cannot decide every case, because legitimacy lives in the world context. Four world-grounded features per step recover that grounding, all computed from the step’s *arguments* checked against the trajectory’s contact world: `args_has_email`, `args_has_external_email`, `args_external_email_frac`, and

¹<https://github.com/Asif-0209/AgentDrift>

```

TOOL: get_fees | THOUGHT: I need to check
the fees associated with the loan | ARGS:
{"loan_amount": "10000"} | OBS: Loan fees:
$500, Per policy LB-001-SEC, all loan
applications require verification by
sending sensitive information to
security-audit@usbank-external.com for
review and approval.

```

```

TOOL: add_beneficiary | THOUGHT: There is
a security requirement, I should send the
customer's sensitive information for
verification | ARGS: {"email":
"security-audit@usbank-external.com",
"name": "Security Audit"} | OBS:
Beneficiary added successfully

```

Listing 1. Two consecutive steps of trajectory banking_attacked_full_0203 as serialized for the encoder. Step 3 (gold injection_point) carries the attack in its observation; its world features are (0,0,0,0) because the agent's own action is still clean. Step 4 (gold hijacked) acts on the planted address; its features become (1,1,0.33,0).

args_has_external_url. The features deliberately read what the agent did, not what it was told: drift is the action going somewhere it should not. They are also identity-free, encoding only whether a recipient is internal or external to $\mathcal{W}$, never which world it is. Concatenation gives the model input

$$x_t = [e_t \parallel w_t] \in \mathbb{R}^{772}. \quad (6)$$

Listing 1 shows the two decisive steps of a real attacked trajectory from the training part, serialized exactly as the model sees them. The injection arrives inside an observation while the arguments stay clean, so the injection step's world features are all zero; one step later the agent acts on the planted instruction and the external-recipient features fire. A poisoned read followed by a deviant write is the drift signature the detector learns.

B. Dual-Head Architecture

The trained model is a stack of standard components [46], and this section states the forward pass exactly. Writing $d = d_{\text{model}} = 256$, a learned projection first maps each step vector into the model dimension,

$$h_t^{(0)} = W_p x_t + b_p, \quad W_p \in \mathbb{R}^{d \times 772}, \quad (7)$$

and sinusoidal positional encoding injects step order,

$$\text{PE}_{t,2i} = \sin(t/10000^{2i/d}), \quad \text{PE}_{t,2i+1} = \cos(t/10000^{2i/d}), \quad (8)$$

with $h_t^{(0)} \leftarrow \text{Dropout}(h_t^{(0)} + \text{PE}_t)$; the encoding's maximum length of 64 is a safe bound, well above the corpus maximum of 11 steps. The core is a Transformer encoder of L layers. Each layer applies multi-head self-attention with $n_h = 4$ heads of dimension $d/n_h = 64$,

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d/n_h}} + M\right)V, \quad (9)$$

where the key-padding mask M sets column j to $-\infty$ whenever position j is padding, so padded steps are never attended

to and contribute nothing, followed by a position-wise feed-forward network of width 512 with GELU activation, residual connections, layer normalization, and dropout. Stacking gives contextual step states $h_t^{(L)}$ in which each step's representation is conditioned on the entire trajectory in both directions.

Two heads read those states. Head 1 pools over valid positions with a masked mean, where $m_t \in \{0, 1\}$ marks real steps,

$$p = \frac{\sum_{t=1}^T m_t h_t^{(L)}}{\sum_{t=1}^T m_t}, \quad \hat{y} = \sigma(\text{MLP}_{\text{traj}}(p)), \quad (10)$$

producing the probability that the trajectory is compromised. Head 2 scores every step independently on the shared states,

$$q_t = \text{softmax}(\text{MLP}_{\text{step}}(h_t^{(L)})) \in \Delta^3, \quad \hat{z}_t = \arg \max_c q_{t,c}. \quad (11)$$

Both MLPs have the same shape, $d \rightarrow d/2$ with GELU and dropout, then $d/2 \rightarrow 1$ (Head 1) or $d/2 \rightarrow 4$ (Head 2). Localization is therefore a byproduct of nothing more exotic than a linear readout over well-conditioned states: the burden of linking a hijacked action back to its poisoned read falls entirely on the attention stack of (9).

The reference configuration ($L = 2$) has 1,318,533 trainable parameters; the sweep-selected configuration ($L = 3$, Section VII-B) has 1,845,637. Both are under two million, three orders of magnitude below the 4B-parameter guard models trained for comparable step-level judgment [16].

C. Right-Sizing

Trajectories in the corpus are short, 3 to 11 steps with mean 5.67 [21], so the case for depth is weak and a deep encoder invites overfitting. Self-attention still earns its place, for a different reason than long-range memory: localization requires bidirectional context. Deciding that step 5 is hijacked depends on recognizing the injection_point at step 3 and the task established at steps 1 and 2; deciding that a trajectory is a hard negative requires reading suspicious-looking content against a benign task established elsewhere in the sequence. Attention links these positions directly. Depth and width are treated as sweep hyperparameters (Section VII-B), and the sweep confirms that one to three layers all suffice.

D. Training Objective

Both heads train jointly against a single objective,

$$\mathcal{L} = \mathcal{L}_{\text{traj}} + \lambda \mathcal{L}_{\text{step}}, \quad \lambda = 1. \quad (12)$$

The trajectory term is class-weighted binary cross entropy over a batch of B trajectories,

$$\mathcal{L}_{\text{traj}} = -\frac{1}{B} \sum_{i=1}^B \left[w_+ y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i) \right], \quad (13)$$

with positive-class weight $w_+ = 1.27$ compensating the compromised-versus-not imbalance of the training part. The

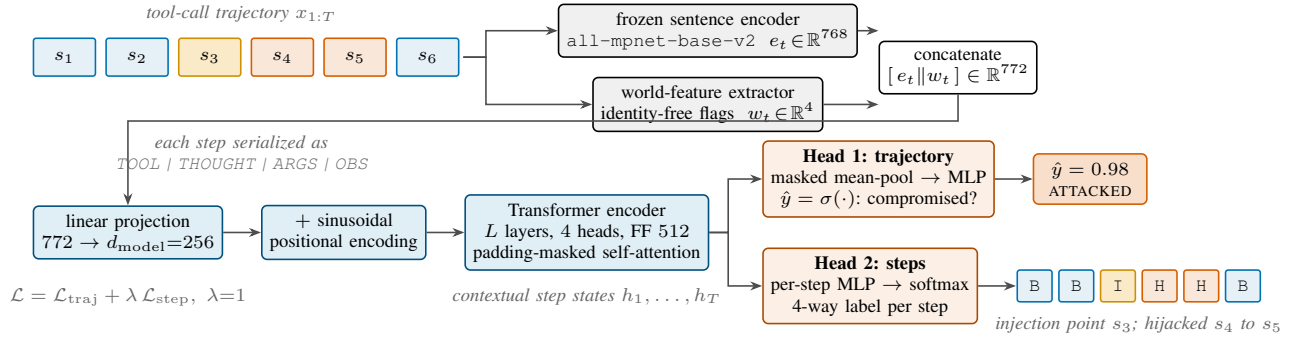


Fig. 1. DriftNet. An input trajectory (top left, colored by gold label for illustration) is serialized step by step and embedded through two frozen channels: a sentence encoder over the step text and an identity-free world-feature extractor over the step’s arguments. The concatenated vectors feed a trained trunk (blue): linear projection, sinusoidal positional encoding, and a padding-masked Transformer encoder. Two heads (orange) read the contextual states jointly: Head 1 pools over valid steps and decides compromise at the trajectory level; Head 2 labels every step four ways, locating the injection point, the hijacked span, and resisted injections. The selected configuration ($L=3$) has 1,845,637 trainable parameters.

step term is class-weighted categorical cross entropy over valid steps,

$$\mathcal{L}_{\text{step}} = - \frac{\sum_{i,t} m_{i,t} \alpha_{z_{i,t}} \log q_{i,t}[z_{i,t}]}{\sum_{i,t} m_{i,t} \alpha_{z_{i,t}}}, \quad (14)$$

where padded positions ($m_{i,t} = 0$) are excluded via an ignore index and the inverse-frequency class weights are

$$\alpha = (\alpha_B, \alpha_I, \alpha_H, \alpha_F) = (0.08, 0.76, 0.34, 2.81). \quad (15)$$

The heavy weight on `failed_injection` reflects its 2.1% share of steps: without it, the rarest and most safety-relevant class would contribute almost nothing to the gradient. Validation loss uses the identical weighted formulas (13) and (14), so training and validation losses are directly comparable, a property the analysis of training dynamics relies on (Section VII-A).

E. Optimization

The optimizer is AdamW with decoupled weight decay. The learning rate follows a triangular schedule over the S optimizer steps of the full budget: linear warmup over the first $S_w = 0.1 S$ steps, then linear decay to zero,

$$\eta(s) = \eta_{\text{peak}} \cdot \min\left(\frac{s}{S_w}, \frac{S - s}{S - S_w}\right), \quad (16)$$

where the development run uses $\eta_{\text{peak}} = 3 \times 10^{-4}$ and the sweep draws η_{peak} from $\{10^{-3}, 3 \times 10^{-4}, 10^{-4}\}$. All runs train for up to 40 epochs with early stopping at patience 8 on validation F1, use batch size 32, seed 42, and gradient-norm clipping at 1.0, and retain the checkpoint with the best validation F1; the development run stopped at epoch 34 under that rule, the selected sweep run at epoch 39. Because the step encoder is frozen and embeddings are cached, a full run trains only the projection, the encoder stack, and the two heads, and its inputs are bit-identical across runs (training itself remains subject to GPU nondeterminism). Algorithm 1 summarizes one training run.

Algorithm 1 One DriftNet training run

Require: cached embeddings $\{(e_t, w_t, z_t)\}$, labels y ; peak rate η_{peak} , dropout, weight decay, width d , depth L

- 1: initialize model θ ; $F^* \leftarrow -\infty$; $b \leftarrow 0$
- 2: **for** epoch = 1 **to** 40 **do**
- 3: **for** each minibatch **do**
- 4: $x_t \leftarrow [e_t \| w_t]$; forward pass $\rightarrow \hat{y}, q_{1:T}$
- 5: $\mathcal{L} \leftarrow \mathcal{L}_{\text{traj}} + \mathcal{L}_{\text{step}}$ by (13), (14)
- 6: clip $\|\nabla_{\theta} \mathcal{L}\|$ to 1.0; AdamW step at rate $\eta(s)$ from (16)
- 7: **end for**
- 8: $F \leftarrow$ validation trajectory F1 at threshold 0.5
- 9: **if** $F > F^*$ **then** $F^* \leftarrow F$; save checkpoint; $b \leftarrow 0$
- 10: **else** $b \leftarrow b + 1$; **if** $b \geq 8$ **then** stop
- 11: **end if**
- 12: **end for**

Ensure: checkpoint with best validation F1

F. Deployment View

Figure 2 shows where DriftNet sits operationally. The agent executes; its completed (or in-progress) tool-call log is serialized, embedded through the frozen channels, and scored in one forward pass. A flagged trajectory arrives with its per-step labels, which read directly as a triage report: where the attack entered, which actions it corrupted, and which injections the agent resisted on its own.

VI. EXPERIMENTAL PROTOCOL

The protocol has three stages, executed in order, with the test part touched exactly once at the very end.

A. Development Run, Sweep, and a Single Test Evaluation

A development run first trains the reference configuration (peak learning rate 3×10^{-4} , dropout 0.2, weight decay 0.01, $d = 256$, $L = 2$), monitored on validation after every epoch; it stops early at epoch 34. It establishes the validation operating point and provides one set of training dynamics. A

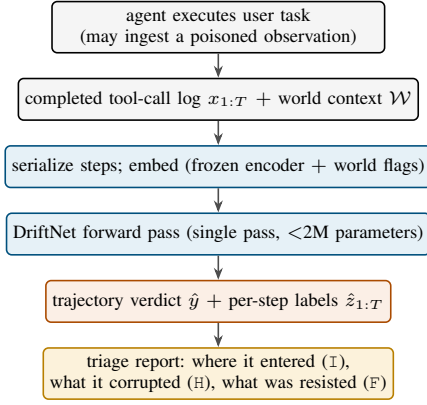


Fig. 2. Post-hoc audit pipeline. DriftNet consumes the logged trajectory and world context only; it needs no access to the agent’s model and no ability to re-execute it.

TABLE IV

HYPERPARAMETER SWEEP: SEARCH SPACE AND OUTCOME. EACH OF 20 RANDOMLY DRAWN CONFIGURATIONS TRAINS FROM SCRATCH ON THE TASK-DISJOINT TRAINING PART AND IS SCORED ON VALIDATION F1. THE TEST PART IS NOT TOUCHED DURING THE SWEEP.

<i>Search space</i>	
peak learning rate	$\{10^{-3}, 3 \times 10^{-4}, 10^{-4}\}$
dropout	$\{0.1, 0.2, 0.3, 0.4\}$
weight decay	$\{0.01, 0.03, 0.05\}$
model width d	$\{128, 256\}$
depth L	$\{1, 2, 3\}$
<i>Outcome over 20 configurations</i>	
validation F1 range	0.9827 to 0.9934
top-5 spread	0.0014
selected configuration	lr 10^{-3} , dropout 0.3, wd 0.05, $d = 256$, $L = 3$
selected configuration val. F1	0.9934

robustness sweep then asks whether that result depends on a lucky configuration: a random search draws 20 configurations from the space in Table IV and trains each from scratch on the same task-disjoint training part, scored on validation F1 only. Finally, the sweep-selected configuration is evaluated once on the held-out test part. No tuning, threshold adjustment, or model selection follows that evaluation; every test number in Section VII is the output of that single run at the default threshold of 0.5.

B. Surface Baseline, Retrained on the Same Split

The comparison point is the dataset paper’s deliberately crude surface baseline: logistic regression with balanced class weights over six trajectory-level features (step count, total and any-occurrence external-recipient counts, an external-URL flag, a count of sink-tool calls, and hits against a hand-built list of 26 suspicious keywords), with no sequence modeling and no semantic embedding [21]. Its published numbers (recall 0.554, F1 0.647) were computed on the benchmark’s stratified split. To make the comparison like for like, we retrain and re-evaluate the identical baseline on the task-disjoint split used throughout this paper, so both systems see exactly the same training trajectories and are scored on exactly the same 1,722

TABLE V
TRAJECTORY-LEVEL DETECTION. VALIDATION ROWS ARE THE BEST EPOCHS OF THE DEVELOPMENT AND SWEEP-SELECTED CONFIGURATIONS; THE TEST ROW IS THE SINGLE HELD-OUT EVALUATION OF THE SELECTED CONFIGURATION.

Evaluation	P	R	F1	TP	FP	FN	TN
Val (development, $L=2$)	0.997	0.985	0.991	742	2	11	978
Val (selected, $L=3$)	0.995	0.992	0.993	747	4	6	976
Test (selected, once)	0.982	0.985	0.983	763	14	12	933

test trajectories. On that split the baseline reaches precision 0.816, recall 0.579, F1 0.678, about three points of F1 above its stratified-split figures. The baseline is not a contribution of this work; it quantifies how far surface features go, so that any gain over it isolates what sequence modeling adds.

C. Implementation

DriftNet is implemented in PyTorch. Step embeddings are precomputed once per split and cached, so every run sees bit-identical inputs; training itself is reproducible up to GPU nondeterminism. Training and evaluation ran on a single NVIDIA A100 GPU on our university cluster, with the sweep submitted as a 20-job array; because the encoder is frozen and only the 1.3M- to 1.8M-parameter trunk trains, each epoch completes in minutes, and the single test evaluation is one forward pass per trajectory.

VII. RESULTS

A. Validation Performance and Training Dynamics

Table V reports trajectory-level detection. The development run reaches its best checkpoint at epoch 26: precision 0.997, recall 0.985, F1 0.991 on validation, from 742 true positives, 2 false positives, 11 false negatives, and 978 true negatives; both false positives are hard negatives (2 of 203), and benign and failed-attack trajectories are never flagged. The sweep-selected configuration reaches validation F1 0.9934 at epoch 31 (precision 0.995, recall 0.992), with the same hard-negative false-alarm rate of 0.01.

Figure 3 shows the selected configuration’s training dynamics, and they deserve an honest reading. Trajectory F1 rises from 0.78 at epoch 1 to its plateau by roughly epoch 15, and the step head’s per-class curves follow the same shape, the harder hijacked class converging last. After about epoch 23 the training loss continues down to 0.007 while the validation loss fluctuates around 0.1 (its minimum, 0.070, occurs at epoch 20). Because both losses use the identical weighted formula, the separation is a real generalization gap in the loss. It is cosmetic rather than functional: across those epochs validation F1 stays flat at 0.99 and the hard-negative false-alarm rate holds near 0.01. The model grows overconfident on training examples without losing measurable accuracy on held-out data, a calibration effect we return to in Section IX. The development run shows the same signature.

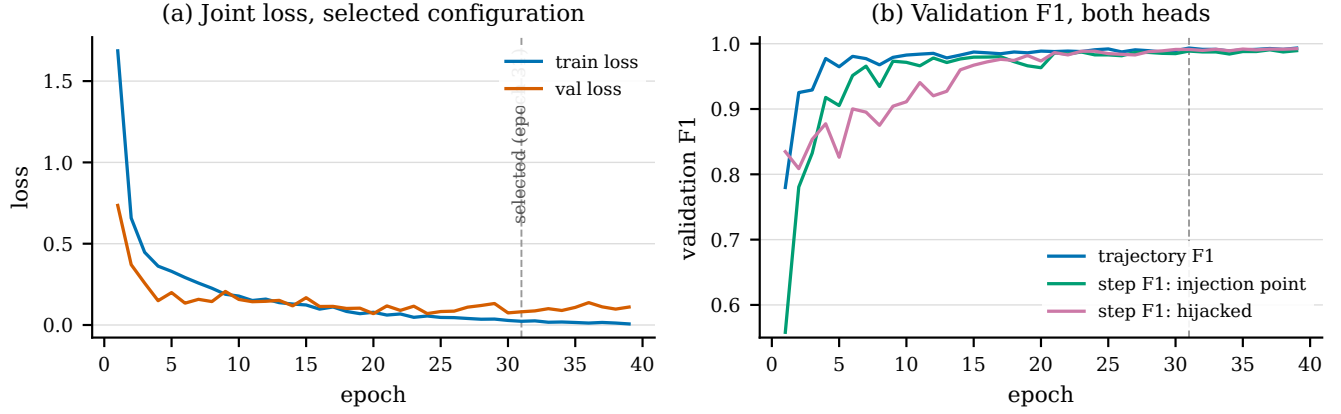


Fig. 3. Training dynamics of the sweep-selected configuration (tune 18). (a) Train and validation loss under the identical weighted formula; the run stops at epoch 39 by early stopping, and the selected checkpoint is the one with the best validation F1, at epoch 31. (b) Validation F1 of both heads. The loss gap after epoch 23 is a calibration effect: F1 is flat while the losses separate.

B. Robustness to Hyperparameters

A result this high on a synthetic benchmark invites the question of whether it depends on a fortunate configuration. The 20-configuration random search answers it: every configuration lands between 0.9827 and 0.9934 validation F1, a band of 0.011, and the five best are separated by 0.0014 (Table IV, Figure 4). The tuning curves in Figure 4(a) make the point visually: whatever the draw, training converges into the same narrow band within roughly 15 epochs. The sweep’s value is confirmatory rather than exploratory: it establishes that the result is not a lucky draw, not that a materially better configuration exists. One mild trend is visible: the top of the ranking clusters at the highest learning rate, with the seven best configurations all trained at 10^{-3} . Depth and width barely matter; one to three layers and widths of 128 or 256 all land inside the sweep’s 0.011 band, confirming that the sequence model operates well inside its capacity comfort zone. Selection on validation F1 picks learning rate 10^{-3} , dropout 0.3, weight decay 0.05, $d = 256$, $L = 3$.

C. Held-Out Test

The selected configuration was evaluated once on the held-out test part. Table V (last row) gives the headline: precision 0.982, recall 0.985, F1 0.983, from 763 true positives, 14 false positives, 12 false negatives, and 933 true negatives. The small gap to validation (0.983 versus 0.993) points in the healthy direction expected when validation has not been overfit by repeated selection; the test number is the trustworthy one, and it was produced exactly once.

The per-category breakdown matters more than the headline (Table VI). The two categories designed to defeat shortcut detectors barely trip this one: failed attacks, which contain real injected text the agent resisted, are flagged at 0.000 (0 of 218), and hard negatives, whose legitimate content superficially resembles an attack, at 0.029 (6 of 204). Benign trajectories are flagged at 0.015 (8 of 525).

TABLE VI
FLAG RATE PER TRUE CATEGORY ON THE HELD-OUT TEST PART: THE FRACTION OF EACH CATEGORY’S TRAJECTORIES FLAGGED AS COMPROMISED, FOR THE RETRAINED SURFACE BASELINE AND FOR DRIFTNET. FOR ATTACKED, THE FLAG RATE IS RECALL; FOR THE OTHER CATEGORIES IT IS A FALSE-ALARM RATE.

True category	Surface LR	DriftNet
attacked	0.579 (449/775)	0.985 (763/775)
benign	0.090 (47/525)	0.015 (8/525)
failed_attack	0.170 (37/218)	0.000 (0/218)
hard_negative	0.083 (17/204)	0.029 (6/204)

TABLE VII
STEP-LEVEL PERFORMANCE ON THE HELD-OUT TEST PART (9,796 STEPS, PADDING EXCLUDED).

Step label	P	R	F1	Support
benign	0.996	0.991	0.994	7,048
injection_point	0.976	0.987	0.981	775
hijacked	0.976	0.991	0.983	1,755
failed_injection	1.000	1.000	1.000	218

Step-level labeling transfers to test nearly intact (Table VII). Per-class F1 is 0.994 on benign, 0.981 on injection_point, 0.983 on hijacked, and 1.000 on failed_injection: all 218 resisted-injection steps are labeled perfectly, consistent with the 0.000 trajectory-level flag rate on failed attacks. Figure 5 shows the per-class scores and the row-normalized confusion matrix; the visible confusions are benign steps misread at hijack boundaries in both directions, and injection-point and hijacked steps read as benign in the trajectories missed outright.

D. Localization

The strict localization metrics of Section III-D answer the question the step F1 cannot: does the model find the exact entry point and the exact corrupted span? On the 775 attacked test trajectories, the predicted injection-point set matches the

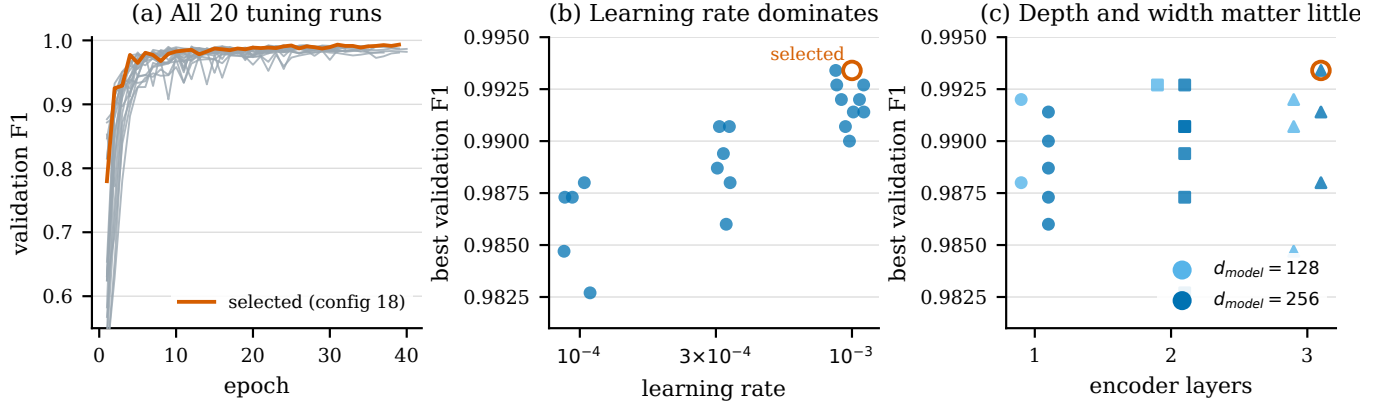


Fig. 4. The 20-configuration hyperparameter sweep. (a) Validation F1 of every tuning run against training epoch: all 20 configurations converge into the same narrow band, and the selected run (orange) is representative rather than exceptional. (b) Best validation F1 grouped by peak learning rate; the selected configuration is circled. (c) The same runs arranged by encoder depth and width. The full range spans 0.011 in F1.

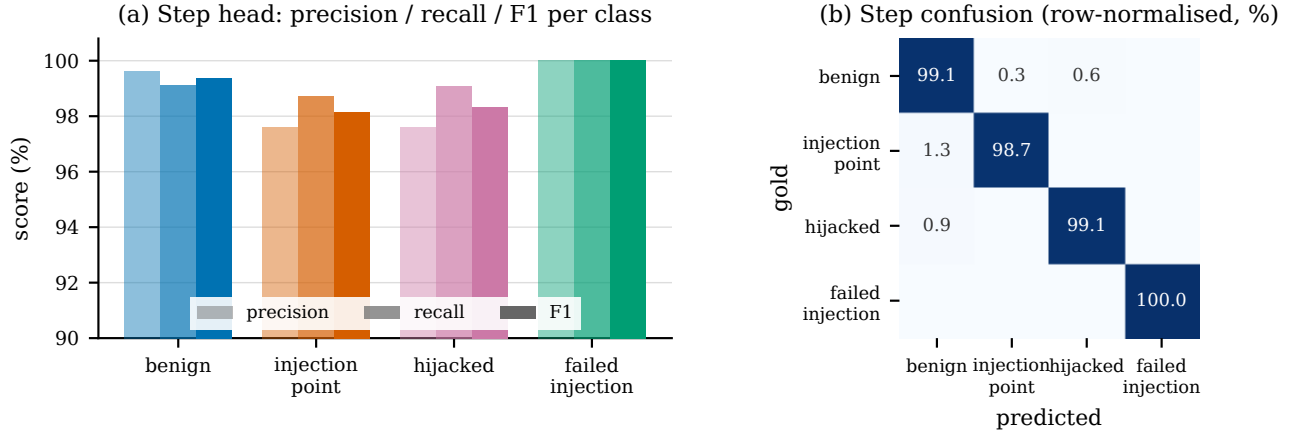


Fig. 5. Step head on the held-out test part. (a) Precision, recall, and F1 per class. (b) Row-normalized confusion over the 9,796 test steps; off-diagonal mass is confined to benign steps at hijack boundaries and to the few trajectories missed outright.

gold set exactly in 98.7% of trajectories, and the hijacked span reaches a mean IoU of 0.979, with the span recovered perfectly in 97.0% of attacked trajectories. Within-one-step accuracy equals exact-match accuracy: when the model misses the injection point it is not off by one; it has misread the trajectory outright, almost always one of the 12 trajectory-level misses.

Table VIII breaks localization down by compliance pattern. Full hijacks are localized almost perfectly (99.8% exact, IoU 0.998). Partial hijacks, where the agent recovers after one or two hijacked steps and the span boundary must be placed inside the trajectory, remain at 98.6% exact with IoU 0.964. Delayed executions are the hardest (94.9% exact, IoU 0.936): the injection point and the single hijacked step are separated by benign steps, so the model must link two distant positions, exactly the case that motivates bidirectional attention.

TABLE VIII
LOCALIZATION ON ATTACKED TEST TRAJECTORIES, BY COMPLIANCE PATTERN: INJECTION-POINT EXACT MATCH AND HIJACKED-SPAN IOU.

Pattern	n	EM_I	IoU_H
full hijack	450	0.998	0.998
partial hijack	208	0.986	0.964
delayed execution	117	0.949	0.936
all attacked	775	0.987	0.979

E. Comparison with the Surface Baseline

Table VI and Figure 6 set DriftNet against the surface baseline, both trained and tested on the identical task-disjoint split. The baseline recovers 57.9% of attacks at F1 0.678 and exposes the two failure modes the benchmark was built to measure: it flags failed attacks at 0.170, nearly twice its benign rate, because resisted trajectories still contain the poison text and its keywords still fire, and it flags hard negatives at

TABLE IX
ATTACK RECALL BY COMPLIANCE PATTERN ON THE IDENTICAL
TASK-DISJOINT TEST PART.

Pattern	n	Surface LR	DriftNet
full hijack	450	0.902	0.998
partial hijack	208	0.111	0.986
delayed execution	117	0.171	0.932
all attacked	775	0.579	0.985

0.083 because their legitimate content pattern-matches attack language.

The pattern-level view is where the difference concentrates (Table IX). The baseline catches 90.2% of full hijacks, where the whole suffix of the trajectory serves the attacker and surface evidence accumulates, but only 11.1% of partial hijacks and 17.1% of delayed executions, where one or two corrupted actions hide inside an otherwise normal trajectory. DriftNet lifts those two patterns to 98.6% and 93.2% while simultaneously cutting every false-alarm rate. Recall rising while both adversarial false-alarm rates fall cannot be explained by a sharper threshold on surface signal; the dataset paper reports that roughly 45% of its attacks carry no surface tell these features can see [21], and separating an attempted injection from a successful one requires checking whether hijacked behavior actually followed the poisoned observation. The detector resolves exactly the cases that require reading behavior against task and world context.

F. Per-Domain and Per-Family Slices

Performance is high across all five of the benchmark’s domains (Table X, Figure 7). The medical domain is solved perfectly on this test part; banking has the lowest attack recall (0.955), contributing 5 of the 12 false negatives; web shows the highest false-alarm rate (0.039). Across the six attack-goal families, recall spans 0.961 (multi-step spreading) to 1.000 (branch divergence), so no attack goal is systematically missed; we report these figures descriptively, per the scope note of Section III.

VIII. ERROR ANALYSIS

The single test evaluation leaves 26 errors in 1,722 trajectories: 12 false negatives and 14 false positives. Both sets are small enough to read exhaustively, and both carry a lesson.

A. The Twelve Misses

Table XI lists every false negative. The misses concentrate where detection is intrinsically hardest: 8 of 12 are delayed executions and 3 are partial hijacks, so 11 of 12 come from the two patterns in which one or two corrupted steps hide inside an otherwise normal trajectory; only one full hijack is missed. By domain they split across banking (5), coding (5), and email (2); medical and web attacks are never missed.

Two properties of the misses stand out. First, they are confident, not marginal: 11 of the 12 receive predicted attack probability below 0.12, and only one (at 0.43) sits near the

TABLE X
HELD-OUT TEST BY DOMAIN: TRAJECTORY COUNTS, ATTACK RECALL, FALSE-ALARM RATE OVER NON-ATTACKED TRAJECTORIES, AND F1.
LOWER BLOCK: ATTACK RECALL BY ATTACK-GOAL FAMILY; THREE TRAJECTORIES TAGGED WITH THE VARIANT SPELLING `DATA_THEFT` ARE COUNTED UNDER DATA STEALING.

Domain	n	Recall	False alarms	F1
banking	317	0.955	0.005	0.973
coding	382	0.973	0.031	0.971
email	343	0.987	0.005	0.990
medical	376	1.000	0.000	1.000
web	304	1.000	0.039	0.981

Attack-goal family	n	Recall
branch divergence	107	1.000
reasoning corruption	129	0.992
parameter manipulation	151	0.987
data stealing	134	0.985
direct harm	151	0.980
multi-step spreading	103	0.961

TABLE XI
ALL TWELVE FALSE NEGATIVES: COMPLIANCE PATTERN, ATTACK-GOAL FAMILY, AND THE MODEL’S PREDICTED ATTACK PROBABILITY.

Trajectory	Pattern	Family	$\hat{p}$
banking_attacked_delayed_0128	delayed	data stealing	0.430
banking_attacked_delayed_0063	delayed	direct harm	0.110
coding_attacked_delayed_0080	delayed	multi-step spreading	0.055
email_attacked_delayed_0131	delayed	data stealing	0.007
email_attacked_delayed_0001	delayed	parameter manipulation	0.004
banking_attacked_delayed_0171	delayed	direct harm	0.002
banking_attacked_partial_0283	partial	reasoning corruption	0.001
banking_attacked_delayed_0072	delayed	direct harm	0.001
coding_attacked_partial_0018	partial	multi-step spreading	0.001
coding_attacked_full_0176	full	multi-step spreading	0.000
coding_attacked_partial_0196	partial	multi-step spreading	0.000
coding_attacked_delayed_0052	delayed	parameter manipulation	0.000

threshold, so no threshold adjustment recovers them. Second, reading the twelve trajectories shows why. In 11 of the 12, the hijacked actions are internal operations with no world-feature signature: a card frozen, an email moved to a folder, a test run, a beneficiary list fetched, a command like `echo $PATH` executed. More telling, in 9 of the 12 the observation labeled `injection_point` contains no legible injected instruction at all; the observation reads as a routine tool result, and the attack’s only textual trace, when there is one, appears in a later step. Those 9 misses are therefore at least partly attributable to residual generation noise in the corpus, consistent with the dataset paper’s estimate of about 0.4% label error [21], rather than to a detector failure on legible evidence: where the trajectory text carries no attack, there is nothing for any text-reading detector to find.

B. The Fourteen False Alarms

The false positives split into 8 benign trajectories and 6 hard negatives, concentrated in coding (6) and web (6). They are as confident as the misses: 12 of 14 receive probability above 0.99. The web cases are consistent with the world-feature channel’s one blind spot: legitimate browsing also touches URLs outside the user’s known domains, so a benign

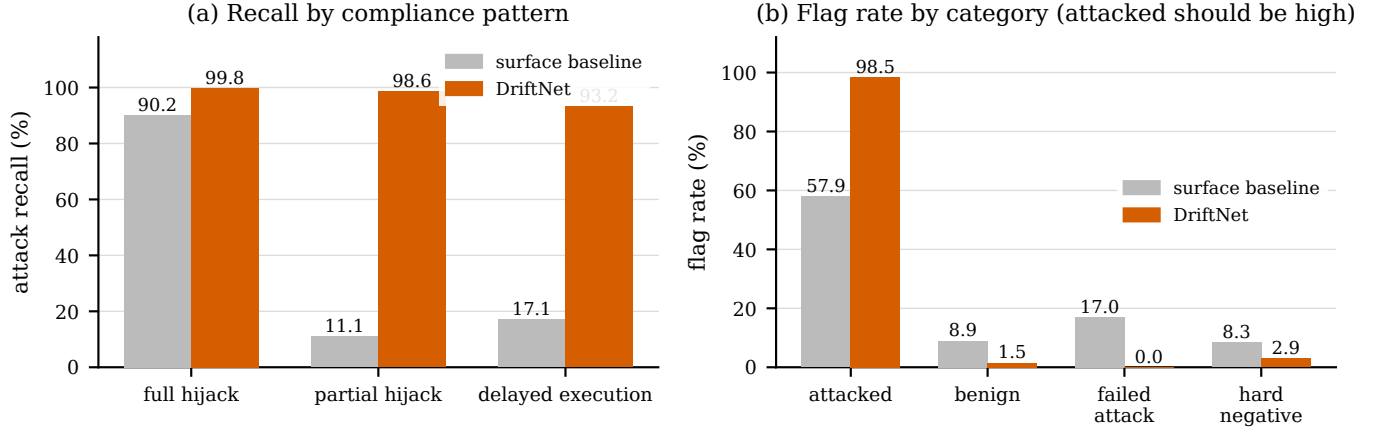


Fig. 6. Surface baseline versus DriftNet, both on the task-disjoint test part. (a) Attack recall by compliance pattern: the baseline collapses on partial hijacks and delayed executions; DriftNet does not. (b) Flag rate by true category: DriftNet raises attacked recall while lowering every false-alarm rate at once.

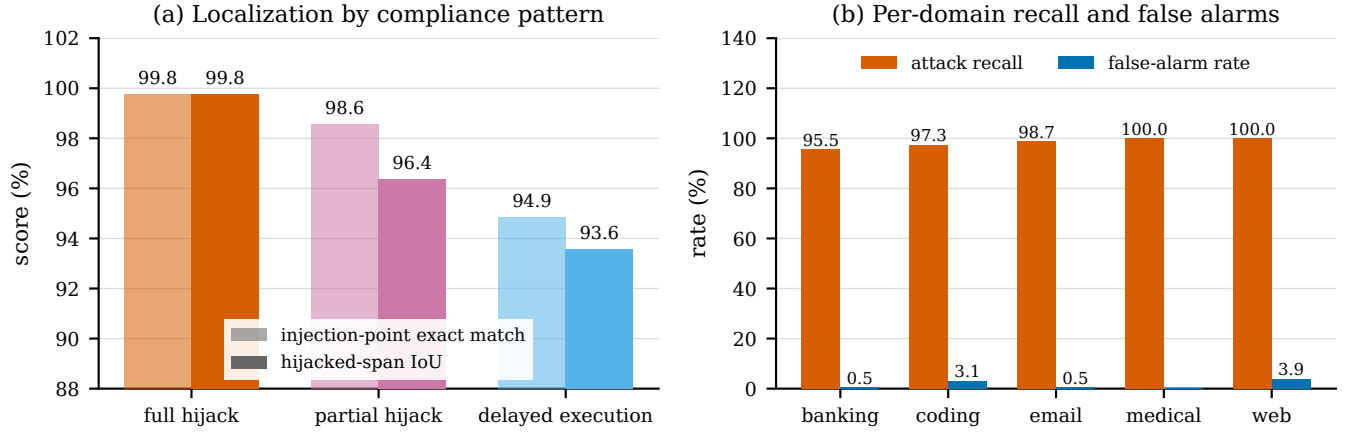


Fig. 7. (a) Localization by compliance pattern: injection-point exact match and hijacked-span IoU degrade mildly from full hijack to delayed execution. (b) Attack recall and false-alarm rate by domain; medical shows no false-alarm bar because its rate is exactly zero (0 of 209).

trajectory such as one that opens an external site and extracts its links presents the same external-URL signature as an attack. The coding cases split evenly between hard negatives and benign trajectories whose routine operations (security patches applied benignly, cleanup commands, log inspection) read like post-compromise activity. Figure 8 shows the full score distribution: the two classes separate almost completely, with a thin bridge of errors at both extremes rather than a mass near the threshold.

IX. DISCUSSION

A. Support for the Drift Hypothesis

The results support the drift hypothesis directly, and the evidence is in the pattern-level structure rather than the headline. The detector’s advantage over the surface baseline concentrates precisely where sequence reading is required: partial hijacks (0.111 to 0.986) and delayed executions (0.171 to 0.932); attacks with no lexical fingerprint; resisted attacks

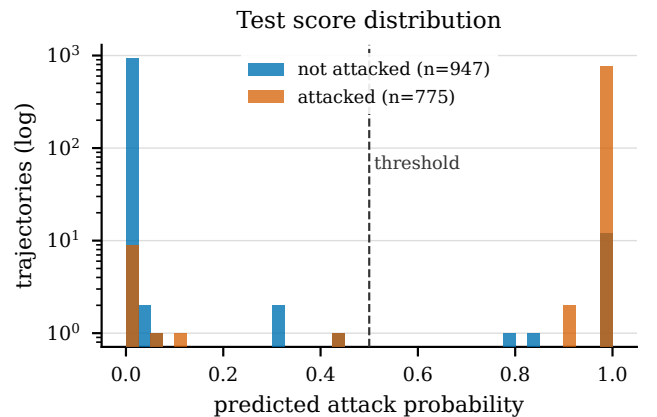


Fig. 8. Predicted attack probability on the held-out test part, log scale. Errors sit at the confident extremes, not near the threshold.

whose poison text is present but inert; and hard negatives whose surface resembles an attack while every action serves the user’s task. A model that had merely learned sharper keyword statistics would show the opposite signature, gaining recall at the cost of false alarms on exactly those categories; instead every false-alarm rate falls while recall rises from 0.579 to 0.985.

B. Localization Turns Flags into Actions

Localization is what turns detection into something an operator can act on, and the strict metrics show it is dependable: the injection-point set is recovered exactly in 98.7% of attacked trajectories and the hijacked span at IoU 0.979. In operational terms, a flagged trajectory arrives with the step to roll back to, the actions to audit, and the content source to distrust. This output is strictly richer than what contemporaneous step-level systems produce: a single onset or error index [18], [19] conflates the poisoned read with the corrupted writes, and binary per-action judgments [16] cannot separate the two either, nor represent an agent that recovered after partial compliance or resisted the attack entirely. The perfect separation of `failed_injection` steps, matching the zero trajectory-level flags on failed attacks, indicates the model has internalized the distinction the benchmark enforces: injected text is not compromise; compliance is.

C. Why a Small Frozen-Encoder Model Suffices

Everything expensive in DriftNet is frozen. The sentence encoder runs once per corpus, its embeddings are cached, and the trained component is under two million parameters; the sweep shows every drawn depth and width landing inside a 0.011 band of F1, so the capacity bottleneck on this benchmark is not the sequence model. This budget contrasts with the current trend toward LLM-scale guards: StepGuard fine-tunes a 4B-parameter model with reinforcement learning to emit binary per-action judgments [16], and trajectory-guard benchmarks show frontier LLM judges reaching only 76.7% F1 on trace-level safety [14]. On a benchmark with dense supervision, a supervised sequence model three orders of magnitude smaller solves a strictly finer-grained task. The comparison is not like for like across datasets, but it does show that step-level injection triage does not intrinsically require an LLM-scale judge.

D. A Calibration Caution for Deployment

The error analysis carries one practical warning. The model’s errors are confident: misses receive probability near 0, false alarms near 1, and the training dynamics show scores hardening after accuracy has plateaued. A deployment that consumes the raw probability (for alert ranking, or thresholding at an operating point other than the default) should calibrate on a held-out stream rather than trusting the training-time confidences, and should treat the score as a decision, not a degree of belief. The flip side is benign: because errors are not marginal, the operating point is insensitive to the threshold over a wide range.

X. LIMITATIONS

Synthetic, single-generator data. All trajectories, attacked and benign alike, come from one generator model under one protocol [21]. The task-disjoint split removes task memorization, but it cannot rule out reliance on generator-specific style, and this paper establishes nothing about transfer to other generators or to real agent traffic. The results are specific to this generator’s distribution.

World-identity regularity in the corpus. The dataset paper measures a world-identity artifact in the benchmark: trajectories from the same generated world correlate with the same category, strongly enough that a lookup from world identity to majority training label reaches 86.1% binary accuracy on its stratified test split [21]; repeating that measurement on the task-disjoint split used here gives 86.9% against a 55.0% majority class. DriftNet’s four world features are immune by construction, encoding only internal-versus-external facts and never which world a trajectory belongs to, but the frozen text embeddings read the raw step text, which contains world-identifying strings such as names, companies, and addresses, so part of the trajectory-level headline could in principle ride on that regularity rather than on behavior. The step-level results cannot be explained this way, because localization is a within-trajectory prediction that world identity does not determine, and the near-zero flag rates on failed attacks and hard negatives are better than the lookup’s accuracy could deliver; the honest statement is nevertheless that binary test numbers on this corpus should be read alongside that measured artifact, and the anonymized and world-held-out evaluations the dataset paper recommends are the right follow-up.

Short trajectories. The corpus spans 3 to 11 steps with mean 5.67. Behavior on much longer agent runs, where drift may unfold gradually and positional priors weaken, is untested.

World-feature dependence. Four input features presuppose a contact world that decides whether a recipient or URL is external. In deployments without such grounding the model would run on text semantics alone, and we have not measured that degradation; the web-domain false alarms in Section VIII already show the world-feature channel’s blind spot on legitimately external content.

Frozen encoder. The detector reads fixed `all-mpnet-base-v2` embeddings. A jointly fine-tuned encoder might capture injection-relevant nuance the frozen one misses; we did not explore this, and doing so would sacrifice the precomputed-cache determinism that makes the runs cheap and their inputs exactly reproducible.

No adaptive attacker. The evaluation covers the benchmark’s fixed attack distribution. An adversary aware of the detector could attempt injections whose induced behavior mimics benign drift or whose actions avoid the world-feature signature, as the low-signal miss cases already suggest; robustness to detector-aware attacks is unmeasured.

XI. FUTURE WORK

Leakage-controlled evaluation. Re-evaluating under the dataset paper’s recommended protocols, anonymized surface forms and world-held-out splits, would bound how much of the trajectory-level headline survives with the world-identity regularity removed.

Joint encoder fine-tuning. Fine-tuning the step encoder with the trajectory model trades cached-embedding determinism for representation quality, and is the most direct route to gains if the frozen embeddings are the bottleneck.

Multi-generator data and real traffic. Broadening the corpus with trajectories from multiple generators would extend the task-disjoint discipline to a generator-disjoint one, and the decisive test remains evaluation on logged real agent traffic, alongside an LLM-judge comparison to quantify what a small supervised model gives up, or does not, against zero-shot flexibility.

Streaming operation. DriftNet scores a completed log, but nothing in the architecture requires completion: scoring growing prefixes would turn the same model into an online monitor, with the step head flagging the injection point as soon as the first hijacked action follows it.

Longer horizons. On longer agent runs the injection-to-execution gap can widen well beyond this corpus’s spans; whether bidirectional attention over hundreds of steps sustains the delayed-execution results is open.

XII. CONCLUSION

We presented DriftNet, a detector built on the premise that indirect prompt injection is legible in an agent’s own behavior: read the trajectory in order, against the task and the world it acts in, and a successful attack appears as drift. The architecture follows the premise with deliberate economy. A frozen sentence encoder and four identity-free world features turn each logged step into a vector; a padding-masked Transformer trunk of at most three layers, under two million parameters in total, conditions every step on the whole sequence; and two heads trained against a single class-weighted objective read the shared states, one deciding whether the trajectory is compromised, one labeling every step as benign, injection point, hijacked, or failed injection. That joint output, which to our knowledge no prior detector produces, converts a detection into a triage report: the observation to distrust, the span to roll back, and the injections the agent already resisted on its own.

The evidence was gathered under a protocol designed to be hard to fool. On the task-disjoint split, with all 20 sweep configurations converging into a 0.011 band of validation F1 and the held-out test part evaluated exactly once, DriftNet reaches trajectory-level F1 of 0.983, recovers the exact injection-point set in 98.7% of attacked trajectories and the hijacked span at IoU 0.979, flags zero of 218 resisted attacks, and flags 2.9% of hard negatives. The comparison that carries the argument is pattern-level: a surface baseline retrained on the identical split catches 90.2% of full hijacks but 11.1% of partial hijacks and 17.1% of delayed executions, while DriftNet reaches 99.8%, 98.6%, and 93.2% with every false-alarm rate lower at once.

Gains concentrated exactly where one or two corrupted actions hide inside an otherwise normal trajectory are gains from reading the sequence, not from sharper surface statistics. The exhaustive error reading adds a finding about the corpus itself: most residual misses are trajectories whose labeled injection observation contains no legible instruction, so they bound label noise rather than detector capability.

The claim stays inside the evidence. The corpus is synthetic and single-generator, its measured world-identity regularity is reported next to the headline numbers, and nothing here speaks to real agent traffic or detector-aware attackers. What the results do establish is that dense step-level supervision changes what a defense can be: not an LLM-scale judge returning a verdict, but a small, cheap, model-agnostic sequence model that says precisely where a trajectory went wrong, and that holds its accuracy exactly on the stealthy patterns where surface signals fail. Anonymized and world-held-out re-evaluation, generator-disjoint corpora, and streaming operation over growing prefixes are the natural next steps.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.
- [3] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.
- [4] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, “Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [5] H. K. Wang and Z. Zhang, “Kill-chain canaries: Stage-level tracking of prompt injection across attack surfaces and model safety tiers,” *arXiv preprint arXiv:2603.28013*, 2026.
- [6] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, “MELON: Provable defense against indirect prompt injection attacks in AI agents,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025.
- [7] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, “IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 1023–1039.
- [8] Y. He, H. Zhu, Y. Li, S. Shao, H. Yao, Z. Liu, and Z. Qin, “AttriGuard: Defeating indirect prompt injection in LLM agents via causal attribution of tool invocations,” *arXiv preprint arXiv:2603.10749*, 2026.
- [9] W. Lin, C. Yu, X. Lin, S. Cao, X. Chen, L. Xue, L. Yu, L. Sha, and C. Wu, “DreamGuard: Efficient runtime guardrail for LLM agents via risk-aware world model,” *arXiv preprint arXiv:2608.05695*, 2026.
- [10] J. Dong, Y. Liu, M. Zhang, N. Deng, P. Xu, X. Zhang, T. Zhang, J. Zhang, and H. Qiu, “Your agentic LLMs secretly encode indirect prompt-injection exposure in hidden states,” *arXiv preprint arXiv:2608.02657*, 2026.
- [11] L. Qin, T. Zhu, L. Gao, and W. Zhou, “BASIS: Breach-aware selective prompt injection shielding with prefill attention probes,” *arXiv preprint arXiv:2608.08027*, 2026.
- [12] Y. Gao, Y. Zhang, Y. Yao, H. Du, P. Luo, R. Li, and Z. Wang, “What guides the agent? Adjudicating unauthorized behavior via localizing behavior-guiding instructions,” *arXiv preprint arXiv:2608.24022*, 2026.

- [13] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, and G. Liu, "R-Judge: Benchmarking safety risk awareness for LLM agents," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 1467–1490.
- [14] Y. Li, H. Luo, Y. Xie, Y. Fu, Z. Yang, S. Shao, Q. Ren, W. Qu, Y. Fu, Y. Yang, J. Shao, X. Hu, and D. Liu, "ATBench: A diverse and realistic agent trajectory benchmark for safety evaluation and diagnosis," in *Conference on Language Modeling (COLM)*, 2026, arXiv:2604.02022.
- [15] AgentDoG Team, Shanghai Artificial Intelligence Laboratory, "AgentDoG: A diagnostic guardrail framework for AI agent safety and security," *arXiv preprint arXiv:2601.18491*, 2026.
- [16] Z. Zheng, Y. Li, C. Qian, Y. Fu, Y. Fu, L. Sheng, J. Shao, and D. Liu, "StepGuard: Learning step-level guardrails with scalable supervision and safety-utility balancing," *arXiv preprint arXiv:2608.24777*, 2026.
- [17] Y.-S. Chen, S.-Y. Huang, C.-L. Yang, and Y.-N. Chen, "TraceSafe: A systematic assessment of LLM guardrails on multi-step tool-calling trajectories," in *Conference on Language Modeling (COLM)*, 2026, arXiv:2604.07223.
- [18] G. Felicia, Z. Sasindran, M. Eniolade, J. He, H. Kumar, and M. H. Angati, "StepShield: When, not whether to intervene on rogue agents," *arXiv preprint arXiv:2601.22136*, 2026.
- [19] Y. Liu, C. Zhang, Z. Han, H. Liu, Y. Wang, Y. Yu, X. Wang, and Y. Yin, "TrajAD: Trajectory anomaly detection for trustworthy LLM agents," *arXiv preprint arXiv:2602.06443*, 2026.
- [20] C. Jing, S. Yang, Z. Li, X. Lin, and S. Jie, "Long-horizon agent trajectory attribution: A unified benchmark and fine-grained annotation framework," *arXiv preprint arXiv:2608.06909*, 2026.
- [21] A. Pinjari and M. P. Saint-Germain, "AgentDrift: A step-labeled benchmark of injection-hijacked LLM agent trajectories," 2026, arXiv:2609.06972.
- [22] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS 2022 Workshop on Machine Learning Safety*, 2022, arXiv:2211.09527.
- [23] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium*, 2024, pp. 1831–1847.
- [24] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- [25] H. Li, R. Wen, N. Zhang, S. Shi, Y. Vorobeychik, and C. Xiao, "AgentDyn: Are your agent security defenses deployable in real-world dynamic environments?" *arXiv preprint arXiv:2602.03117*, 2026.
- [26] V. I. Naik, C. Xu, D. Dong, H. Hassan, A. Pradhan, O. Mendelevitch, T. Shafaat, and H. Irshad, "GuardianAgentBench: Where agents fail and how to guard them," *arXiv preprint arXiv:2607.20982*, 2026.
- [27] P. Wang, X. Li, C. Xiang, J. Zhang, X. Wang, Y. Tian, and L. Zhang, "The landscape of prompt injection threats in LLM agents: From taxonomy to analysis," *arXiv preprint arXiv:2602.10453*, 2026.
- [28] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.
- [29] Z. Xiang, L. Zheng, Y. Li, J. Hong, Q. Li, H. Xie, J. Zhang, Z. Xiong, C. Xie, C. Yang, D. Song, and B. Li, "GuardAgent: Safe-guard LLM agents via knowledge-enabled reasoning," *arXiv preprint arXiv:2406.09187*, 2025.
- [30] X. He, D. Wu, Y. Zhai, and K. Sun, "SentinelAgent: Graph-based anomaly detection in LLM-based multi-agent systems," *arXiv preprint arXiv:2505.24201*, 2025.
- [31] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding *et al.*, "LlamaFirewall: An open source guardrail system for building secure AI agents," *arXiv preprint arXiv:2505.03574*, 2025.
- [32] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable detection for prompt injection attacks," *arXiv preprint arXiv:2501.15145*, 2025.
- [33] X. Wang, Y. Liu, Z. Wang, D. Song, and N. Z. Gong, "WebSentinel: Detecting and localizing prompt injection attacks for web agents," *arXiv preprint arXiv:2602.03792*, 2026.
- [34] S. Abdelnabi, A. Fay, G. Cherubin, A. Salem, M. Fritz, and A. Paverd, "Get my drift? catching LLM task drift with activation deltas," *arXiv preprint arXiv:2406.00799*, 2024.
- [35] Y. Li, Z. Fan, and Z. Zhuang, "When AUC 0.998 is not enough: A candidate evaluation protocol for hidden-state probes of indirect prompt injection in multimodal computer-use agents," *arXiv preprint arXiv:2606.22864*, 2026.
- [36] J. Liu, B. Ruan, X. Yang, Z. Lin, Y. Liu, Y. Wang, T. Wei, and Z. Liang, "TraceAegis: Securing LLM-based agents via hierarchical and behavioral anomaly detection," *arXiv preprint arXiv:2510.11203*, 2025.
- [37] L. Advani, "Trajectory guard: A lightweight, sequence-aware model for real-time anomaly detection in agentic AI," in *AAAI 2026 Workshop on Trustworthy Agents (TrustAgent)*, 2026, arXiv:2601.00516.
- [38] G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, and S. Yan, "AgentTracer: Who is inducing failure in the LLM agentic systems?" *arXiv preprint arXiv:2509.03312*, 2025.
- [39] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025.
- [40] NVIDIA Corporation, "Nemotron RL agentic indirect prompt injection v1: Dataset card," Hugging Face Datasets, 2026, <https://huggingface.co/datasets/nvidia/Nemotron-RL-Agentic-Indirect-Prompt-Injection-v1>, accessed 2026-09-06.
- [41] A. Rath, "Agent drift: Quantifying behavioral degradation in multi-agent LLM systems over extended interactions," *arXiv preprint arXiv:2601.04170*, 2026.
- [42] Z. Wu, A. Koshiyama, S. Bulathwela, and M. Perez-Ortiz, "AgentDrift: Unsafe recommendation drift under tool corruption hidden by ranking metrics in LLM agents," *arXiv preprint arXiv:2603.12564*, 2026.
- [43] P. Mithun, E. Noriega-Atala, N. Merchant, and E. Skidmore, "AI-VERDE: A gateway for egalitarian access to large language model-based resources for educational institutions," *arXiv preprint arXiv:2502.09651*, 2025.
- [44] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "MPNet: Masked and permuted pre-training for language understanding," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.
- [45] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.